CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY (CHARUSAT)

Chandubhai S. Patel Institute of Technology (CSPIT)

Department of Computer Science & Engineering

ITUE203 – Web Development Frameworks

PRACTICAL 7

PHP Form Processing with Server-Side Validation and Stored Inquiries

PHP 8 · MySQLi prepared statements · MariaDB 10.4 · HTML5 · CSS3

Corrected submission – implemented in the integrated StudentHub portal (studenthub/)

Student Name	Shreyas
Roll Number	25CS023
Student ID	25CS023
Email	25cs023@charusat.edu.in
Course	ITUE203 – Web Development Frameworks
Programme	B.Tech Computer Science & Engineering
Semester / Academic Year	3 / 2026-27
Date of Submission	
Faculty Signature	

Corrected report. This report replaces the original Practical 7 report (`Practical-7/PRAC_7_ITUE203_25CS023.docx`, kept unchanged in the repository). The original report described the standalone application in `Practical-7/` (`index.php` and `records.php`, storing submissions in a JSON file protected by `flock()`). This version describes the same server-side form processing on the portal's Help & Contact page `studenthub/contact.php`: valid inquiries are stored in the database table `inquiries` (migration 005) instead of a file, and administrators read and resolve them in the Admin Console. All screenshots in this report were taken from the canonical integrated application `studenthub/` with Microsoft Edge on disposable test databases with fictional records; all test results are copied from the saved test outputs of the repository.

1. Aim and Objectives

To process the StudentHub inquiry (Help & Contact) form with PHP: receive the data with the POST method, validate and sanitise every field on the server, store valid inquiries safely, show clear success and error messages, and let the administrators work through the stored inquiries.

- Build an accessible PHP form whose data is sent with POST and processed only on the server.
- Validate empty and invalid inputs on the server – name, e-mail address, optional enrollment number, topic and message – without relying on JavaScript or browser checks.
- Sanitise the input and escape every value when it is displayed, so submitted text can never run as HTML.
- Store valid inquiries safely, also when several are sent at the same moment (in the portal: a database table instead of a file).
- Show a clear success message with a reference number after saving, and a message next to each invalid field.
- Intermediate extension: display the stored records – here as the *Inquiries* tab of the Admin Console.
- Advanced extension: protect the form with a CSRF token – here the portal's session token, checked on every POST.

2. Problem Definition

Official problem definition: “Process submitted registration/contact form data using PHP. Validate and sanitize inputs on the server side and store records in CSV or JSON file format. Display success/error messages.”

Practical 7 was first built as a standalone PHP application in `Practical-7/` that appended valid submissions to `private/submissions.json` under an exclusive `flock()` lock. The integrated portal already had a Help & Contact page, but according to the audit in `TEST_REPORT.md` (section 12) “inquiries were **not stored** (only an audit line when signed in)”. The integration, as recorded there, stores validated inquiries in a new table `inquiries` (migration 005), gives the sender a reference number, adds a honeypot field and adds an *Inquiries* tab to the Admin Console; the validation and storage functions are in `studenthub/includes/inquiries.php`.

Storage format. The practical asks for CSV or JSON file storage. Inside the portal, which already keeps all its data in `studenthub_db`, the inquiries are stored in a database table instead. `database/README.md` explains the choice: “Practical 7 kept inquiries in a JSON file protected by `flock()`; in the portal the database provides the same safety for simultaneous submissions (tested with 8 parallel requests).” The JSON-file version with file locking remains unchanged in `Practical-7/` and is described in the original report.

The practical list asks four key questions; the answers for the integrated implementation are:

Key question	Answer in the integrated portal
Is the form submitted using POST?	Yes. The form has `method="POST"` and `action="contact.php"`; the data is processed only when `$_SERVER["REQUEST_METHOD"]` is POST. A GET request only displays the form.
Are inputs validated and sanitized server side?	Yes. `validate_inquiry()` accepts only UTF-8 strings, trims every value, collapses spaces in the name, lower-cases the e-mail and upper-cases the enrollment number, and checks the name pattern and length, the e-mail with `filter_var()`, the enrollment-number format, the topic list and the message length (10–2000 characters). The form has `novalidate` and no validation script, so every check is made by PHP.
Is file writing handled safely?	In the portal no file is written. Each inquiry is one `INSERT` with a prepared statement; the InnoDB table gives every row its own AUTO_INCREMENT id, so simultaneous inquiries cannot overwrite each other – 8 parallel inquiries were all stored with different numbers. The table also enforces the topic list (ENUM) and the message length (CHECK).
Are success and error responses displayed clearly?	After saving, the page redirects (HTTP 303) and shows, once, a message with the sender's name and the reference number, e.g. “Thank you, Neha Rao. Your inquiry has been received by the A7 CSE helpdesk (reference #1).” (Figure 2). Invalid input returns HTTP 422 with “Please correct the highlighted fields. Your inquiry was not sent.”, a message under each invalid field and the entered values kept.

3. Tools and Technologies

Tool / technology	Use in this practical
PHP 8.2.12 (XAMPP), MySQLi	`contact.php`, `includes/inquiries.php`, `admin_dashboard.php`: processing, validation, sessions; prepared statements for INSERT, SELECT and UPDATE
MariaDB 10.4.32	Table `inquiries` in `studenthub_db` (migration `database/migrations/005_inquiries.sql`)
HTML5, CSS3	Form markup with labels and ARIA attributes; `global.css` for field messages and the hidden honeypot field
Bootstrap 5.3.2 (CSS)	Table, badge and button styles of the Admin Console
Python 3 (standard library)	`tests/test_forms.py` (HTTP tests) and `database/tests/test_005_inquiries.py`
Playwright + Microsoft Edge (headless)	`tests/browser_forms.py`: browser checks and the screenshots in Section 8
Git / GitHub	Version control; the integration is commit `938e74f` of the StudentHub repository
Operating system	Windows 11

4. Place in the Integrated StudentHub Portal

The form is the portal's *Help & Contact* page, linked in the *Campus* group of the sidebar and from the home page (“Contact Us”). It is public: visitors can send an inquiry without an account. For a signed-in user the name, e-mail address and enrollment number are filled in from the linked `students` record, and the stored inquiry is linked to the account.

File / route	Role
`studenthub/contact.php` (route `contact.php`)	Form, POST processing, messages; public (`optional_user()`).
`studenthub/includes/inquiries.php`	`INQUIRY_TOPICS`, `validate_inquiry()`,

	`save_inquiry()`, `list_inquiries()`, `inquiry_counts()`, `resolve_inquiry()`.
`studenthub/admin_dashboard.php?tab=inquiries`	Admin Console *Inquiries* tab: list, filter, mark as resolved.
`studenthub/includes/auth.php`	CSRF token functions, `e()`, `redirect()` (HTTP 303), the permissions `admin.console` and `inquiries.manage` (administrators only).
`database/migrations/005_inquiries.sql` / `.down.sql`	Creates (or removes) the `inquiries` table.
`database/studenthub_demo.sql`	Fresh installation; contains the `inquiries` table and records migrations 001–005.

Connections to the other integrated practicals: the CSRF token and sessions come from the integrated login (Practicals 9 and 10); every stored and resolved inquiry is written to the audit log and the Inquiries tab sits in the Admin Console of Practical 14; the `inquiries` table references `users` in the shared portal database `studenthub_db`, into which Practicals 8–15 are integrated step by step (`database/README.md`).

5. Implementation

5.1 The form

All values are printed with `e()`. The form keeps the field names of the earlier portal form (`name`, `id`, `mail`, `topic`, `msg`). When a field is invalid, PHP adds `aria-invalid="true"` and `aria-describedby` and prints the message in a `<span class="field-error">` below it. The last field, `website`, is the honeypot: it is moved off-screen by CSS, hidden from screen readers (`aria-hidden="true"`) and skipped by the Tab key (`tabindex="-1"`).

studenthub/contact.php (lines 91–98) – start of the inquiry form: POST, CSRF token and the first field

```
 91              <form action="contact.php" method="POST" novalidate>
 92                  <?= csrf_field() ?>
 93                  <fieldset>
 94                      <legend class="sr-only">Inquiry Form</legend>
 95                      <p>
 96                          <label for="name">Your Name:</label>
 97                          <input type="text" id="name" name="name" maxlength="100" autocomplete="name"
    value="<?= e($values["name"]) ?>" required<?= inquiry_attrs($errors, "name") ?>>
 98                          <?= inquiry_error($errors, "name") ?>
```

The enrollment number, e-mail, topic and message fields follow the same pattern (lines 100–123); the topic options are generated from `INQUIRY_TOPICS`. The honeypot field is a single line, followed by the two buttons:

studenthub/contact.php (lines 124–127) – the honeypot field and the buttons

```
124                     <p class="hp-field" aria-hidden="true"><label for="website">Leave this field
    empty</label><input type="text" id="website" name="website" tabindex="-1" autocomplete="off"></p>
125                     <p>
126                         <button type="submit"><i class="bi bi-send" aria-hidden="true"></i> Send
    Inquiry</button>
127                         <button type="reset">Clear</button>
```

5.2 POST processing (Post/Redirect/Get)

When the request is a POST, the CSRF token is checked first (`require_valid_csrf()`, Section 7). The input is then validated; the submitted strings are kept for redisplay. A valid inquiry is stored, an audit entry is written, the success message is put into the session (`flash()`) and the browser is redirected back to `contact.php` with HTTP 303, so refreshing the page cannot send the inquiry again. Invalid input returns HTTP 422 and the form is shown again.

studenthub/contact.php (lines 16–28) – processing of the submitted form

```
16  if ($_SERVER["REQUEST_METHOD"] === "POST") {
17      require_valid_csrf();
18      [$clean, $errors] = validate_inquiry($_POST);
19      foreach (INQUIRY_FIELDS as $field) {
20          $values[$field] = is_string($_POST[$field] ?? null) ? $_POST[$field] : "";
21      }
22      if (!$errors) {
23          $inquiry_id = save_inquiry($conn, $clean, $current_user["id"] ?? null);
24          log_audit($conn, $current_user["id"] ?? null, $current_user["username"] ?? "guest",
    "SUBMIT_INQUIRY", "Inquiry #$inquiry_id stored (topic: {$clean["topic"]})", "SUCCESS");
25          flash("success", "Thank you, {$clean["name"]}. Your inquiry has been received by the A7 CSE
    helpdesk (reference #$inquiry_id).");
26          redirect("contact.php");
27      }
28      http_response_code(422);
```

5.3 Server-side validation and sanitising

studenthub/includes/inquiries.php (lines 11–63) – validate_inquiry()

```
11  function validate_inquiry(array $input): array
12  {
13      $errors = [];
14      $raw = [];
15      foreach (INQUIRY_FIELDS as $field) {
16          $value = $input[$field] ?? '';
17          if (!is_string($value) || !mb_check_encoding($value, 'UTF-8')) {
18              $errors[$field] = 'This field contains characters that are not allowed.';
19              $value = '';
20          }
21          $raw[$field] = trim($value);
22      }
23      $clean = [
24          'name'          => (string) preg_replace('/\s+/u', ' ', $raw['name']),
25          'email'         => strtolower($raw['mail']),
26          'enrollment_no' => $raw['id'] === '' ? null : strtoupper($raw['id']),
27          'topic'         => $raw['topic'] === '' ? 'General' : $raw['topic'],
28          'message'       => str_replace("\r\n", "\n", $raw['msg']),
29      ];
30      if (!isset($errors['name'])) {
31          if ($clean['name'] === '') {
32              $errors['name'] = 'Please enter your name.';
33          } elseif (mb_strlen($clean['name'], 'UTF-8') < 2 || mb_strlen($clean['name'], 'UTF-8') > 100)
    {
34              $errors['name'] = 'The name must be 2–100 characters long.';
35          } elseif (!preg_match("/^\\p{L}[\\p{L}\\p{M} .'-]*$/u", $clean['name'])) {
36              $errors['name'] = 'Use letters, spaces, apostrophes, dots and hyphens only.';
37          }
38      }
39      if (!isset($errors['mail'])) {
40          if ($clean['email'] === '') {
41              $errors['mail'] = 'Please enter your email address.';
42          } elseif (strlen($clean['email']) > 100 || filter_var($clean['email'], FILTER_VALIDATE_EMAIL)
    === false) {
43              $errors['mail'] = 'Please enter a valid email address, e.g. name@example.com.';
44          }
45      }
46      if (!isset($errors['id']) && $clean['enrollment_no'] !== null && !preg_match('/^[0-9]{2}[A-
    Z]{2}[0-9]{3}$/', $clean['enrollment_no'])) {
47          $errors['id'] = 'Use the enrollment number format 2 digits, 2 letters, 3 digits (e.g.
    25CS023), or leave it empty.';
48      }
49      if (!isset($errors['topic']) && !in_array($clean['topic'], INQUIRY_TOPICS, true)) {
50          $errors['topic'] = 'Please choose a topic from the list.';
51      }
52      if (!isset($errors['msg'])) {
53          $length = mb_strlen($clean['message'], 'UTF-8');
54          if ($length === 0) {
55              $errors['msg'] = 'Please enter your message.';
56          } elseif ($length < 10 || $length > 2000) {
57              $errors['msg'] = 'The message must be 10–2000 characters long.';
58          }
59      }
60      if (($input['website'] ?? '') !== '') {                 // honeypot: invisible to people, often
```

```
    filled in by bots
61          $errors['msg'] = 'The inquiry could not be accepted.';
62      }
63      return [$clean, $errors];
```

Field	Sanitising	Rule	Message
Your Name (`name`)	trimmed; runs of white space become one space	required; 2–100 characters; `/^\p{L}[\p{L}\p{M} .'-]*$/u`	Please enter your name. / The name must be 2–100 characters long. / Use letters, spaces, apostrophes, dots and hyphens only.
Enrollment Number (`id`, optional)	trimmed, upper-cased; empty becomes NULL	`/^[0-9]{2}[A-Z]{2}[0-9]{3}$/`	Use the enrollment number format 2 digits, 2 letters, 3 digits (e.g. 25CS023), or leave it empty.
Email (`mail`)	trimmed, lower-cased	required; at most 100 characters; `filter_var(…, FILTER_VALIDATE_EMAIL)`	Please enter your email address. / Please enter a valid email address, e.g. name@example.com.
Topic (`topic`)	trimmed; empty becomes General	one of General, Academics, Events, Portal access, Other	Please choose a topic from the list.
Message (`msg`)	trimmed; CR LF line breaks become LF	required; 10–2000 characters (`mb_strlen`)	Please enter your message. / The message must be 10–2000 characters long.
Every field	–	must be a string with valid UTF-8 (an array such as `msg[]` is refused)	This field contains characters that are not allowed.
Honeypot (`website`)	–	must be empty	The inquiry could not be accepted. (shown under the message)

Sanitising here means normalising the value that is stored; HTML is not removed or encoded before storage. Instead, every value is escaped at the moment it is printed (Section 7), so the stored text stays exactly what the sender wrote.

5.4 Storing and reading inquiries

`save_inquiry()` inserts one row with a prepared statement and returns the new id, which is shown to the sender as the reference number. `user_id` is the id of the signed-in account, or NULL for a visitor. `list_inquiries()` reads the newest inquiries (at most 100), optionally only those with the status `new` or `resolved`, and joins `users` to show the account name.

studenthub/includes/inquiries.php (lines 66–71) – save_inquiry() (list_inquiries() follows in lines 74–87)

```
66  function save_inquiry(mysqli $db, array $clean, ?int $user_id): int
67  {
68      $stmt = $db->prepare('INSERT INTO inquiries (user_id, name, email, enrollment_no, topic, message)
    VALUES (?, ?, ?, ?, ?, ?)');
69      $stmt->bind_param('isssss', $user_id, $clean['name'], $clean['email'], $clean['enrollment_no'],
    $clean['topic'], $clean['message']);
70      $stmt->execute();
71      return (int) $stmt->insert_id;
```

5.5 Displaying the stored records – Admin Console Inquiries tab (intermediate extension)

The Admin Console (`admin_dashboard.php`) is open only to administrators (`admin.console`). Its *Inquiries* tab shows a badge with the number of new inquiries, the counts of new and stored inquiries (e.g. “2 new, 2 in total”), a filter (All / New / Resolved) and a table with the columns #, Received, From, Topic, Message, Status and Action. Every value is escaped with `e()`; the message keeps its line breaks (`white-space: pre-wrap`). An open inquiry has a *Resolved* button – a small POST form with the CSRF token; a resolved one shows the time it was resolved. The markup is the block “INQUIRIES TAB” in `admin_dashboard.php` (lines 1190–1232); Figures 3 and 4 show it rendered.

Marking an inquiry as resolved is accepted only as a POST with a valid CSRF token and the permission `inquiries.manage`. `resolve_inquiry()` changes only an inquiry whose status is still `new` (`UPDATE … WHERE id = ? AND status = 'new'`), so resolving twice gives the message "That inquiry was not found or is already resolved."

studenthub/admin_dashboard.php (lines 168–177) – the resolve action

```
168      // Help & Contact inquiries (Practical 7): mark as resolved – admins only (inquiries.manage)
169      if ($get_action === "resolve_inquiry") {
170          require_permission("inquiries.manage");
171          if ($target_id > 0 && resolve_inquiry($conn, $target_id)) {
172              log_audit($conn, $current_user["id"], $current_user["username"], "RESOLVE_INQUIRY",
     "Inquiry #$target_id marked as resolved", "SUCCESS");
173              flash("success", "Inquiry #$target_id was marked as resolved.");
174          } else {
175              flash("error", "That inquiry was not found or is already resolved.");
176          }
177          redirect("admin_dashboard.php?tab=inquiries");
```

6. Database and Relationships

Migration 005 adds one table to `studenthub_db` and changes nothing else:

database/migrations/005_inquiries.sql (lines 35–51) – the inquiries table

```
35  CREATE TABLE IF NOT EXISTS inquiries (
36      id            INT(11)      NOT NULL AUTO_INCREMENT,
37      user_id       INT(11)      DEFAULT NULL,
38      name          VARCHAR(100) NOT NULL,
39      email         VARCHAR(100) NOT NULL,
40      enrollment_no VARCHAR(20)  DEFAULT NULL,
41      topic         ENUM('General', 'Academics', 'Events', 'Portal access', 'Other') NOT NULL DEFAULT
    'General',
42      message       TEXT         NOT NULL,
43      status        ENUM('new', 'resolved') NOT NULL DEFAULT 'new',
44      created_at    TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
45      resolved_at   DATETIME     DEFAULT NULL,
46      PRIMARY KEY (id),
47      KEY idx_inquiries_status (status, created_at),
48      KEY idx_inquiries_user (user_id),
49      CONSTRAINT fk_inquiries_user FOREIGN KEY (user_id) REFERENCES users (id) ON DELETE SET NULL ON
    UPDATE CASCADE,
50      CONSTRAINT chk_inquiries_message CHECK (CHAR_LENGTH(message) BETWEEN 10 AND 2000)
51  ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Element	Meaning
`id` INT AUTO_INCREMENT, primary key	The reference number shown to the sender (#1, #2, …).
`user_id` → `users.id`, `fk_inquiries_user`	Optional link to the signed-in account (NULL for visitors). ON DELETE SET NULL: deleting an account keeps its inquiries; ON UPDATE CASCADE.
`topic` ENUM	The same five values as `INQUIRY_TOPICS` in PHP; default General.
`message` TEXT + CHECK `chk_inquiries_message`	`CHAR_LENGTH(message) BETWEEN 10 AND 2000` – the PHP rule repeated in the database.
`status` ENUM('new', 'resolved')	Default new; `resolved_at` is set when an administrator resolves the inquiry.
Indexes	`idx_inquiries_status (status, created_at)` for the filter, `idx_inquiries_user (user_id)` for the foreign key.

Relationship: users 1 : 0..n inquiries through the optional `inquiries.user_id`. The table is InnoDB with `utf8mb4_unicode_ci`, like the other portal tables.

- **Preconditions:** the script stops with a message if no database is selected, if migration 004 is not recorded, or if a different `inquiries` table already exists.

- **Verification and record:** after creating the table it checks that `fk_inquiries_user` references `users` with the delete rule SET NULL, and records version `005` in `schema_migrations`.
- **Rollback:** `005_inquiries.down.sql` drops the table – and with it every stored inquiry – and removes the `005` record.
- **Live database:** according to `database/README.md`, migration 005 was applied to the live database on 31 Oct 2026 after a verified backup; the read-only check `--verify-live` passed 4 / 4.
- **Why PHP validation still matters:** the database README notes that XAMPP runs MariaDB without strict mode, so values that break no CHECK (for example a too-long text) would be silently adjusted; PHP therefore checks every length first.

7. Validation and Security

7.1 CSRF token (advanced extension)

The portal creates one random token per session (`bin2hex(random_bytes(32))`, renewed at login), puts it into every POST form with `csrf_field()` and compares it with `hash_equals()`. A POST without a valid token is stopped with HTTP 403 and written to the audit log as `CSRF_REJECTED` with the status `SECURITY_BLOCK`; nothing is stored.

studenthub/includes/auth.php (lines 95–110) – CSRF token check (the token is created by csrf_token(), lines 82–88)

```
 95  function csrf_is_valid($submitted): bool
 96  {
 97      $expected = $_SESSION['csrf_token'] ?? null;
 98      return is_string($expected) && is_string($submitted) && $submitted !== '' &&
     hash_equals($expected, $submitted);
 99  }
100
101  // For POST handlers: a missing or wrong token stops the request (HTTP 403) and is written to the
     audit log
102  function require_valid_csrf(): void
103  {
104      if (csrf_is_valid($_POST['csrf_token'] ?? null)) {
105          return;
106      }
107      global $conn;
108      log_audit($conn, isset($_SESSION['user_id']) ? (int) $_SESSION['user_id'] : null,
     $_SESSION['username'] ?? 'guest',
109          'CSRF_REJECTED', 'POST to ' . basename($_SERVER['SCRIPT_NAME'] ?? '') . ' without a valid
     token', 'SECURITY_BLOCK');
110      studenthub_fail(403, 'Security check failed', 'This form has expired or was not sent from
     StudentHub. Go back, reload the page and try again. Nothing was changed.');
```

7.2 Other measures

- **Escaping on output:** `e()` is `htmlspecialchars()` with `ENT_QUOTES | ENT_SUBSTITUTE` and UTF-8. It is used for the values shown again in the form, for the success message and for every cell of the Admin Console. Tested with `<b>Bold</b>` as a name and a stored message containing `<script>alert(1)</script>`: both appear as text.
- **SQL injection:** all queries with user input use prepared statements; `inquiry_counts()` runs a fixed query without input.
- **Honeypot:** the hidden `website` field is invisible to people; a simple bot that fills every field is refused with HTTP 422 and nothing is stored.
- **Type and encoding checks:** arrays and invalid UTF-8 are refused in every field.
- **Access control:** the Admin Console requires `admin.console`, the resolve action `inquiries.manage` (both administrators only); a GET request with `action=resolve_inquiry` changes nothing; students and editors receive HTTP 403.

- **Audit and privacy:** a stored inquiry writes `SUBMIT_INQUIRY` (details: "Inquiry #… stored (topic: …)") and resolving writes `RESOLVE_INQUIRY`. The audit entry does not contain the message text. The `inquiries` table stores no IP address; like every audit entry of the portal, the audit row records the IP address and browser string.
- **Post/Redirect/Get:** the success message is stored in the session, shown once after the 303 redirect, and a page refresh cannot repeat the submission.

7.3 Accessibility of the form

Every control has a visible label (`for`/`id`), and the fields are grouped in a `fieldset` with the screen-reader legend "Inquiry Form". Invalid fields get `aria-invalid="true"` and `aria-describedby` pointing to their message; the general error box has `role="alert"` and the success box `role="status"`. The honeypot is hidden from screen readers and from keyboard navigation.

8. Output Screenshots

All screenshots are genuine full-page captures in Microsoft Edge (headless, driven by Playwright, viewport 1280 px wide), made by `tests/browser_forms.py` on 31 October 2026 against the PHP built-in server and a disposable database. The senders "Neha Rao" (visitor) and "Riya Shah" (student account `riya_shah`) and the administrator `portal_admin` are fictional test data.

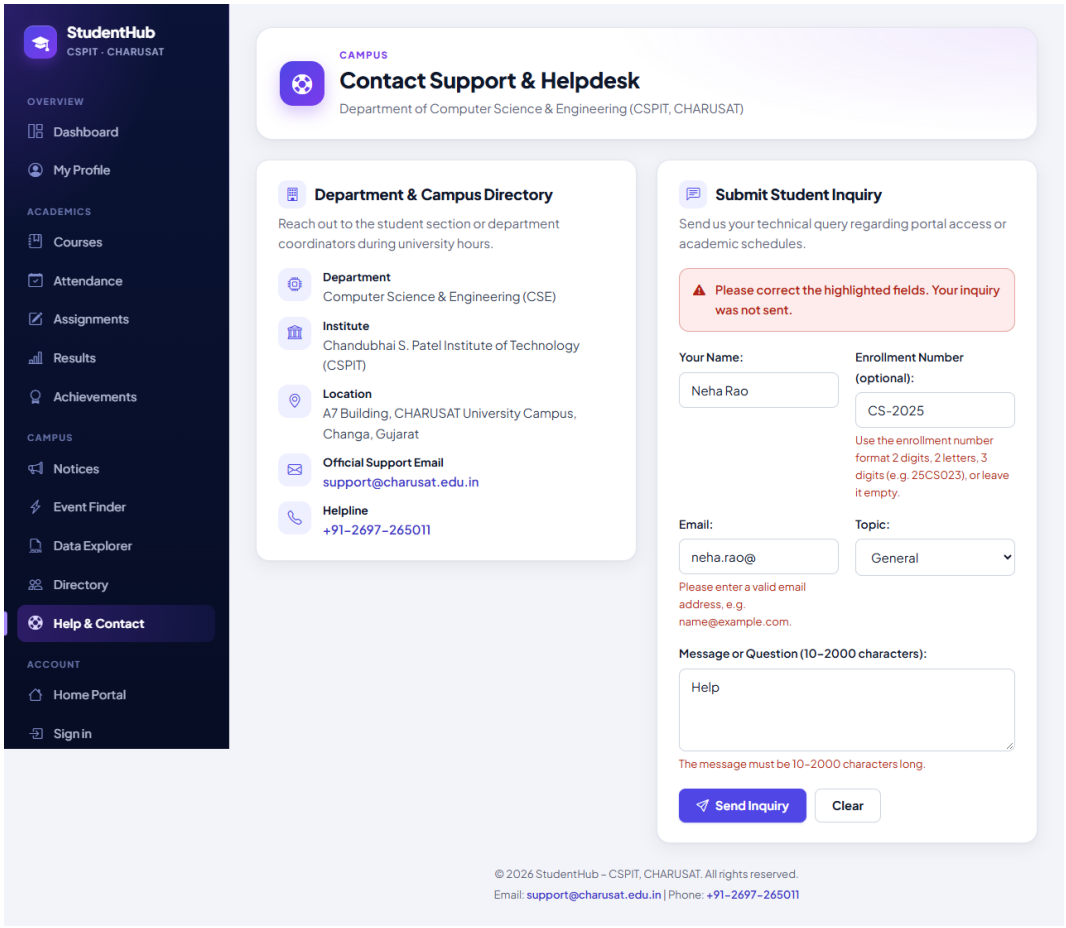


Figure 1: Server-side validation: a visitor submitted name Neha Rao, enrollment number CS-2025, e-mail neha.rao@ and the message "Help". The page shows "Please correct the highlighted fields. Your inquiry was not sent." and a message under the enrollment number, the e-mail and the message; all entered values are kept. The test confirmed that nothing was stored. (08-contact-validation.png)

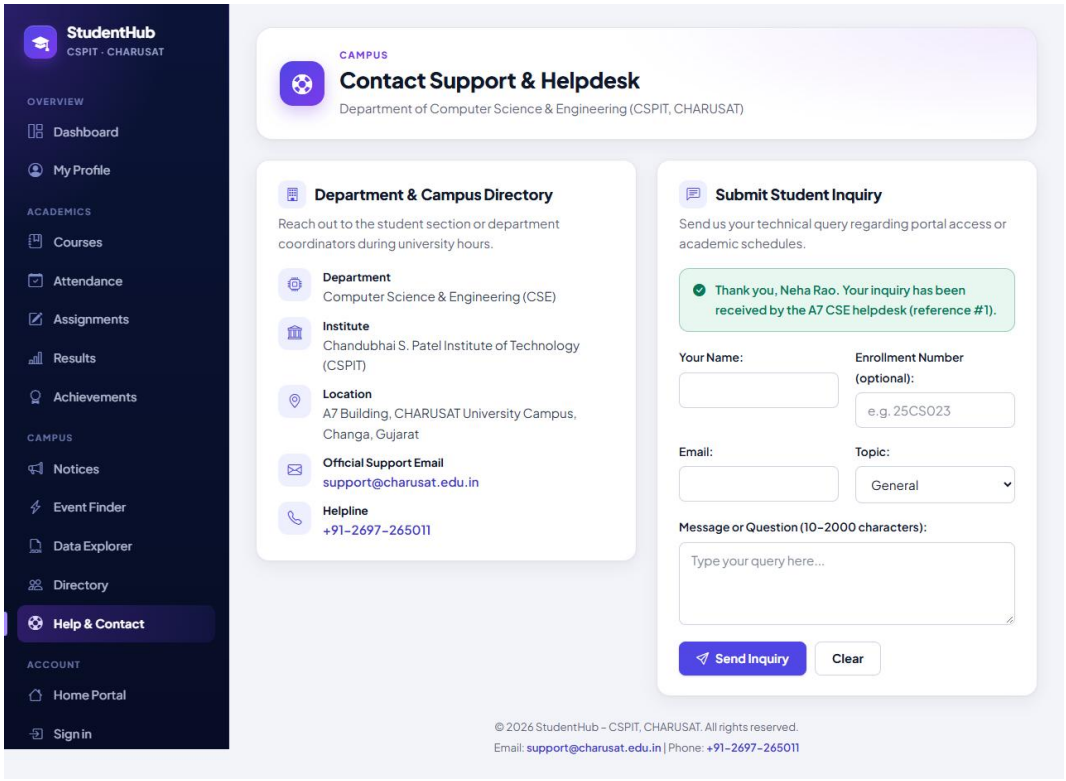


Figure 2: After correcting the e-mail, leaving the enrollment number empty, choosing the topic Portal access and writing a longer message: following the 303 redirect the page shows "Thank you, Neha Rao. Your inquiry has been received by the A7 CSE helpdesk (reference #1)." and an empty form. (09-contact-success.png)

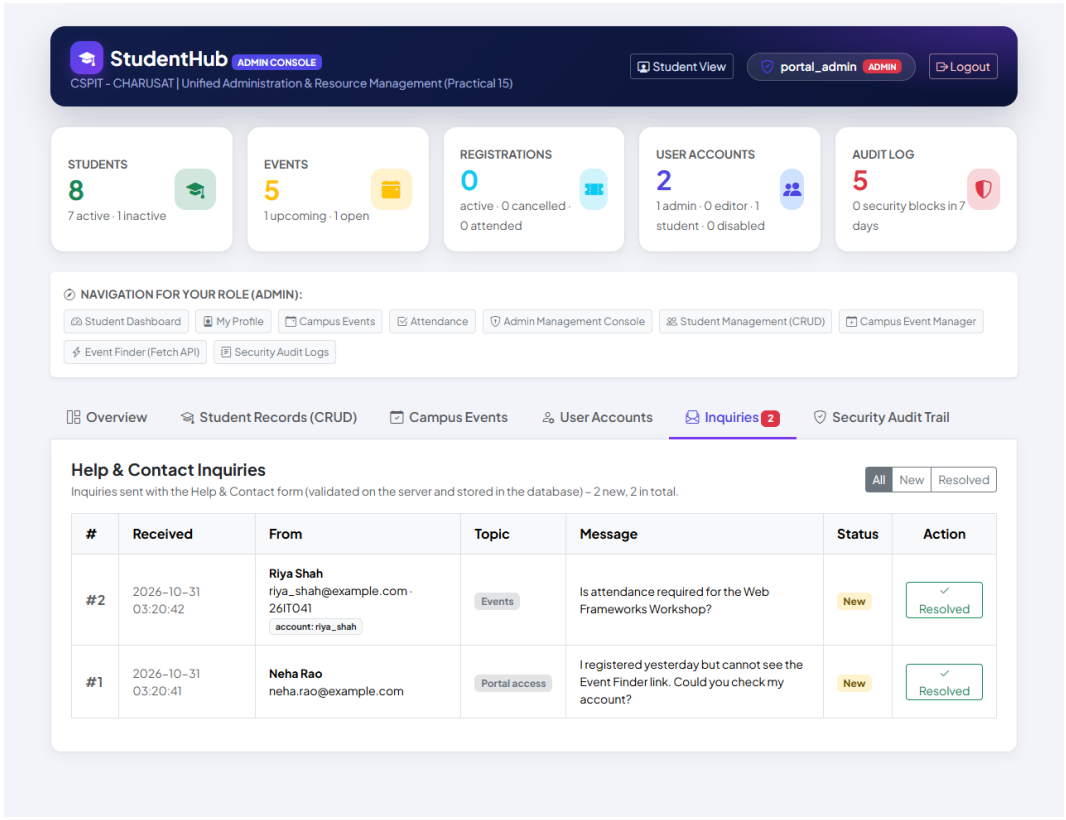


Figure 3: Admin Console, Inquiries tab (badge 2, "2 new, 2 in total"), newest first: #2 from the signed-in student Riya Shah (riya_shah@example.com · 26IT041, badge "account: riya_shah", topic Events) and #1 from the visitor Neha Rao (topic Portal access); both have the status New and a Resolved button. (10-admin-inquiries.png)

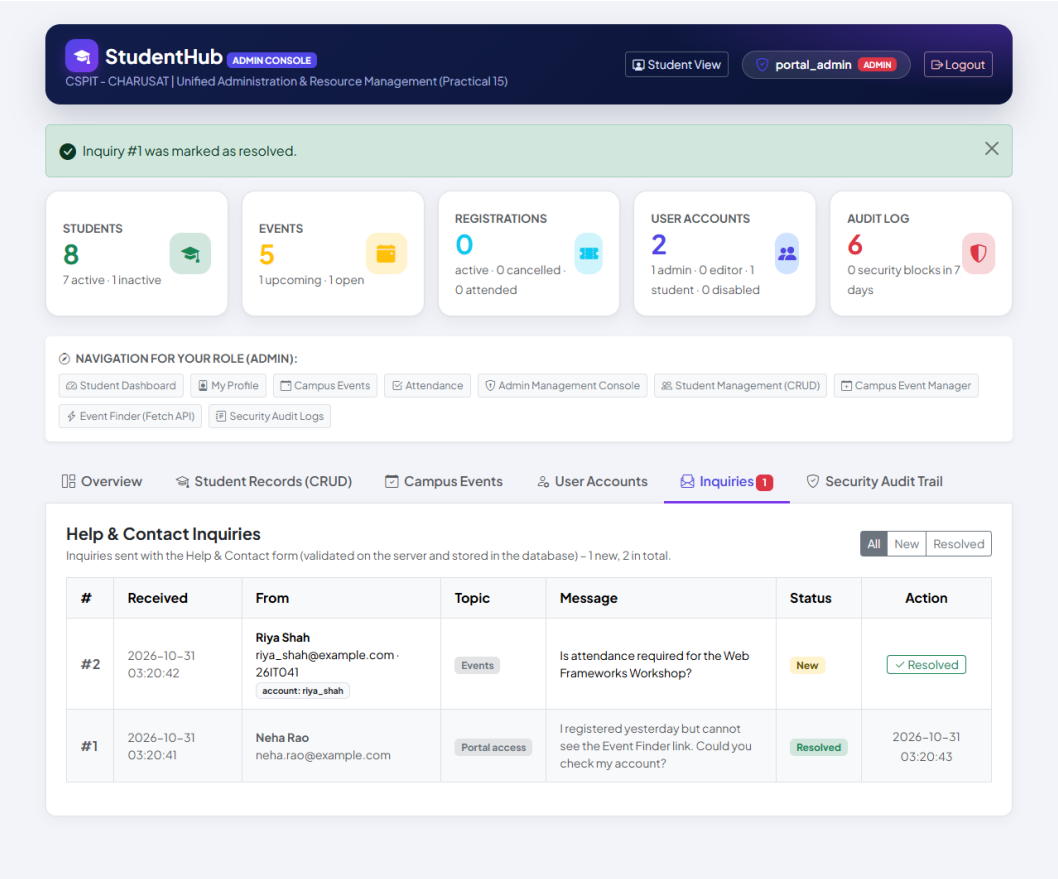


Figure 4: After clicking Resolved for #1: the message "Inquiry #1 was marked as resolved.", the tab badge shows 1 ("1 new, 2 in total"), and row #1 shows the status Resolved with the time 2026-10-31 03:20:43 instead of the button. (11-inquiry-resolved.png)

9. Testing and Results

Environment of the recorded runs (31 October 2026): Windows 11, PHP 8.2.12 (XAMPP) built-in server on 127.0.0.1, MariaDB 10.4.32, Microsoft Edge in headless mode through Playwright. All suites work on disposable databases; `test_forms.py` confirms at the end that the live `studenthub_db` was not touched, and the migration test reads the live database only with `--verify-live`. The complete run of all suites (`tests/run_all_output.txt`) passed 841 checks with 0 failures.

Suite	Scope	Result	Output file
`tests/test_forms.py`	HTTP tests of Practicals 5–7; 19 checks of the contact form and 9 of the Admin Console Inquiries tab concern Practical 7	**41 / 41 passed**	`tests/test_forms_output.txt`
`tests/browser_forms.py`	Microsoft Edge checks and screenshots; 4 checks concern Practical 7, two more cover the	**15 / 15 passed**	`tests/browser_forms_output.txt`

	whole run (no unexpected JavaScript errors, no PHP warnings)		
`database/tests/test_005_inquiries.py` (fixture)	Migration 005 on a fixture with migrations 001–004	**14 / 14 passed**	`database/tests/test_005_output.txt`
`test_005_inquiries.py --source-dump`	Migration 005 on a real dump in a disposable database (result recorded in `TEST_REPORT.md` section 12)	**14 / 14 passed**	–
`test_005_inquiries.py --verify-live`	Read-only comparison of the live database with the baseline upgraded with 005	**4 / 4 passed**	`database/tests/test_005_verify_live_output.txt`

Key checks, summarised from the saved outputs (all PASS):

- **Form and storage:** contact.php is public with a CSRF token, a topic list and a hidden honeypot field; a valid visitor inquiry -> 303 back to the form, stored with the e-mail in lower case, the enrollment number in upper case, status new and no account; the success message names the reference number once; SUBMIT_INQUIRY audit entry without the message text.
- **Refused input (HTTP 422 with the matching message):** empty name, name with digits, invalid e-mail, wrong enrollment format, unknown topic, message under 10 or over 2000 characters, field sent as an array, honeypot filled in; rejected input is shown again HTML-escaped; nothing was stored; without the CSRF token -> 403, nothing stored.
- **Signed-in student and concurrency:** fields prefilled from the linked record and the inquiry linked to the account (user_id); 8 simultaneous inquiries -> all 8 stored with different reference numbers.
- **Admin Console:** newest first with a 'new' badge count; messages HTML-escaped; CSRF token in every POST form; GET or a POST without token changes nothing (403); mark as resolved stores the time and a RESOLVE_INQUIRY entry; resolving again -> error message; filter 'Resolved' works; student and editor -> 403.
- **Migration 005:** only the new table changes; FK ON DELETE SET NULL; existing rows unchanged; repeatable; CHECK refuses a short message; deleting an account keeps its inquiry; rollback and re-apply; precondition stop; fresh install identical.
- **Edge:** the four steps of Figures 1–4 and no JavaScript errors other than the intended 422 response (the browser's automatic `/favicon.ico` request got 404 – a known, pre-existing item: the portal has no favicon).

9.1 Not tested

- Running the portal under Apache (all recorded runs used PHP's built-in server).
- Screen readers, a physical keyboard, browsers other than Edge and narrow (mobile) widths.

- A database failure while an inquiry is being stored.

10. Differences from the Original Standalone Practical and Limitations

Aspect	Standalone Practical-7 (original report)	Integrated portal
Application	`index.php` (form + processing) and `records.php`; PHP built-in server on port 8607 with `router.php`	`studenthub/contact.php` in the portal, plus the Admin Console tab
Fields	full name, e-mail, category (5), message 10–1000 characters	name, optional enrollment number, e-mail, topic (5), message 10–2000 characters
Storage	`private/submissions.json` under `flock(LOCK_EX)`; IDs SUB-0001, SUB-0002, …; damaged files backed up	table `inquiries` (migration 005); ids #1, #2, …; prepared INSERT; ENUM, CHECK and foreign key
Display of stored records	`records.php`, a local demonstration page for everyone	Admin Console *Inquiries* tab, administrators only, with a filter and "mark as resolved"
CSRF	own token, renewed after every successful submission; HTTP 400 on failure	the portal's session token (renewed at login); HTTP 403 and an audit entry on failure
Bot protection	none	honeypot field `website`
Signed-in users	– (no login)	fields prefilled from the student record; inquiry linked to the account
Tests	63 / 63 checks (`tests/test_form.py`)	19 + 9 HTTP checks, 4 Edge checks, migration tests 14 / 14, 14 / 14, 4 / 4

10.1 Limitations

- **Storage format:** the inquiries are stored in a database table, not in a CSV or JSON file as the problem definition asks; the file-based version remains in `Practical-7/`.
- **No e-mail:** the portal sends no e-mail – `DEPLOYMENT.md` states that inquiry answers are not implemented. The helpdesk can only mark an inquiry as resolved.
- **No delete or reopen:** the console offers only "mark as resolved"; it lists at most the newest 100 inquiries.
- **Simple bot protection:** the honeypot stops only simple bots; there is no limit on the number of inquiries per sender.
- **No client-side checks:** the contact page has HTML5 attributes but `novalidate` and no validation script, so errors are shown only after the page has been sent to the server.
- **Rollback:** undoing migration 005 deletes every stored inquiry; export them first if they are needed.

11. Conclusion

The StudentHub Help & Contact form is now processed entirely by PHP inside the integrated portal. Data is sent with POST and a CSRF token, sanitised and validated on the server, and every valid inquiry is stored in the `inquiries` table of `studenthub_db` with a prepared statement – 8 simultaneous inquiries were all stored with distinct reference numbers. Senders see a success message with the reference number after a 303 redirect, or a general message and field-level messages with their input kept; every displayed value is escaped, and a honeypot field refuses simple bots.

Both extensions are integrated: the stored records are displayed in the Admin Console's Inquiries tab, where administrators filter them and mark them as resolved (intermediate), and CSRF protection guards both the form and the resolve action (advanced). The integration passed the HTTP suite (41 / 41), the Edge suite (15 / 15) and the migration tests (14 / 14, 14 / 14 and 4 / 4). The one deliberate deviation from the problem definition – database storage instead of a CSV/JSON file – is explained in Section 2 and Section 10.

12. References

- ITUE203 Web Development Frameworks – Practical List 2026-27, Practical 7: “PHP Form Processing with Server-Side Validation and CSV/JSON File Storage”, CSPIT, CHARUSAT.
- StudentHub repository: https://github.com/TheEmperor08/StudentHub-ITUE203-25CS023 – folders `studenthub/` (portal), `database/` (migration 005 and its tests), `Practical-7/` (original standalone version), `tests/`, `evidence/practical-05-07-integration/`; `TEST_REPORT.md` section 12; `database/README.md`.
- PHP Manual – Dealing with forms: https://www.php.net/manual/en/tutorial.forms.php
- PHP Manual – filter_var and validate filters: https://www.php.net/manual/en/filter.filters.validate.php
- PHP Manual – htmlspecialchars: https://www.php.net/manual/en/function.htmlspecialchars.php
- PHP Manual – mysqli prepared statements: https://www.php.net/manual/en/mysqli.quickstart.prepared-statements.php
- MariaDB Knowledge Base – CONSTRAINT (CHECK and FOREIGN KEY): https://mariadb.com/kb/en/constraint/
- OWASP Cheat Sheet – Cross-Site Request Forgery Prevention: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html